RMIT Classification: Trusted

COSC2626/2640
Virtualization and Hypervisors

Dr. Qiang Fu
School of computing technologies

RMIT UNIVERSITY

House Keeping

- AWS invite sent out
- Assignment 1 is out: due in Week 4, midnight Friday 29/03
  - Answer the questions based on **your understanding** and using **your own words**
- You may have up to 4 members in a group or do it individually
  - The grouping is to help you form groups for the AWS project (Assignment 2).

Recap

- Cloud computing is about …???

Recap

- Sharing of infrastructure, platform, software, ….
  - IaaS, PaaS, SaaS

Service models

IaaS
(infrastructure as a service)

PaaS
(platform as a service)

SaaS
(software as a service)

More control
over IT resources

Less control
over IT resources

This Lecture

- Introduction to virtualization and virtual machines
- Introduction to hypervisors and a bit on operating systems
- Virtualization implementation
  - Trap and emulate
  - ….

Virtualization

- Why do we want virtualization?

- Think about the computers/servers that a company has to maintain
  - Web server
  - Email server
  - Ecommerce server
  - Each on a different computer
- Good or bad?
  - Great isolation and fault tolerance
  - Expensive and difficult to manage

Virtualization

- Can we achieve something similar but in a better way?
  - Virtual Machines (VM)
  - One server for one VM

Virtual Machines

- Efficient sharing of resources while maintaining isolation
- Can be outsourced to Cloud Service Providers (CSPs)
  - A relief to ITS, but ….
  - Security, privacy, vendor lock-in

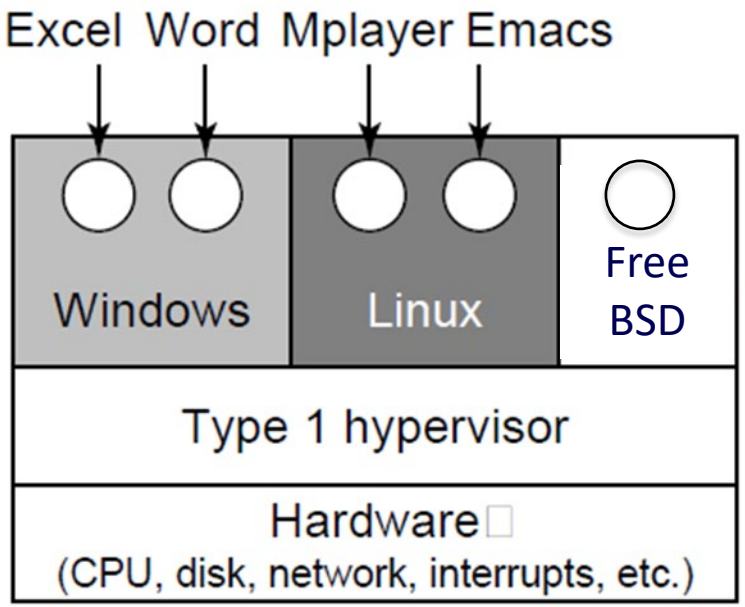
VM overview

- Fundamental idea
  - Abstract hardware of a single computer into several different execution environments (**virtual machine**, or **VM**)
- Several components
  - **Host**: underlying hardware system
  - **Virtual machine manager** (**VMM**) or **hypervisor** creates and runs VMs
  - **Guest**: VM, process provided with virtual copy of the host

- Single physical machine can run multiple operating systems concurrently, each in its own VM

VM overview

- A VM with type 1 hypervisor



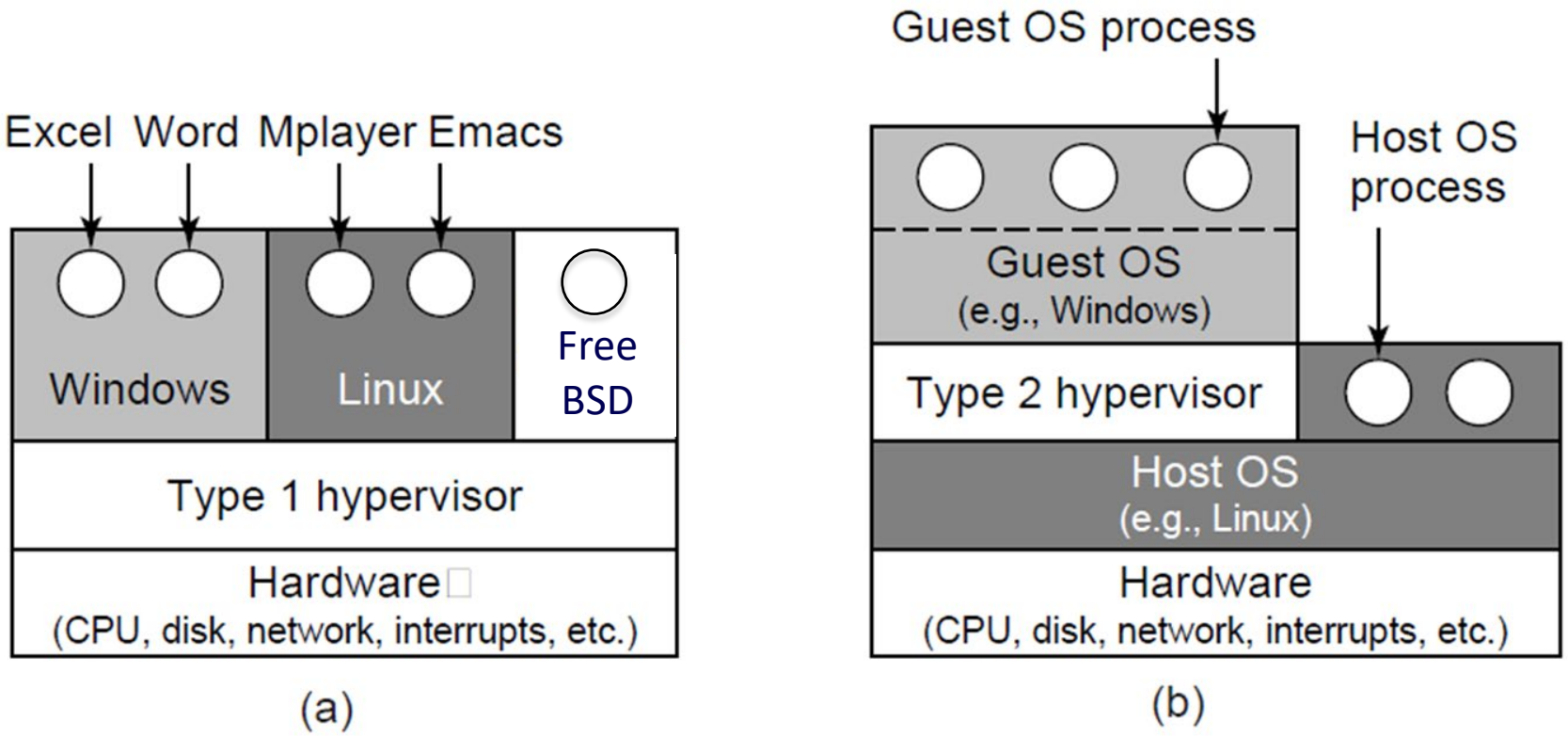
Hypervisor is the key

Hypervisors

- Hypervisors should score well in three dimensions:
  - **Safety (resource control)**: hypervisor should have full control of virtualized resources.
  - **Fidelity (equivalence)**: behaviour of a program on a virtual machine should be identical to same program running on bare hardware.
  - **Efficiency**: much of code in virtual machine should run without intervention by hypervisor.

Hypervisors

Type 1 and type 2 hypervisors.



Location of type 1 and type 2 hypervisors.

Hypervisors

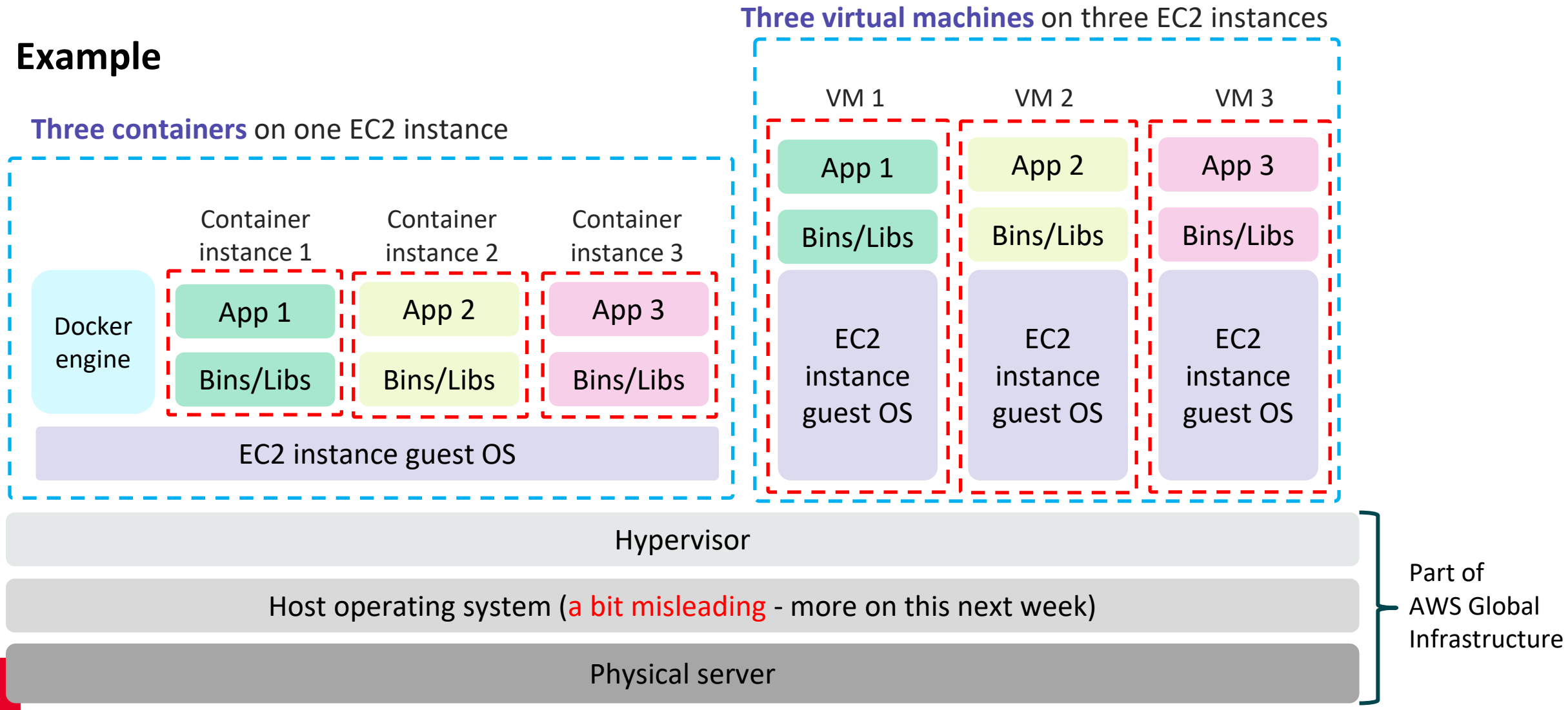
- Type 1 hypervisors (category 1): Operating-system-like software built to provide virtualization
  - VMware ESXi (ESX) and Citrix XenServer
- Type 1 hypervisors (category 2): Also includes general-purpose operating systems that provide standard functions as well as VMM functions
  - Microsoft Windows Server with Hyper-V and RedHat Linux with KVM

Hypervisors

- Type 2 hypervisors: Applications that run on standard operating systems but provide VMM features to guest operating systems
  - VMware Workstation and Fusion and Oracle VirtualBox

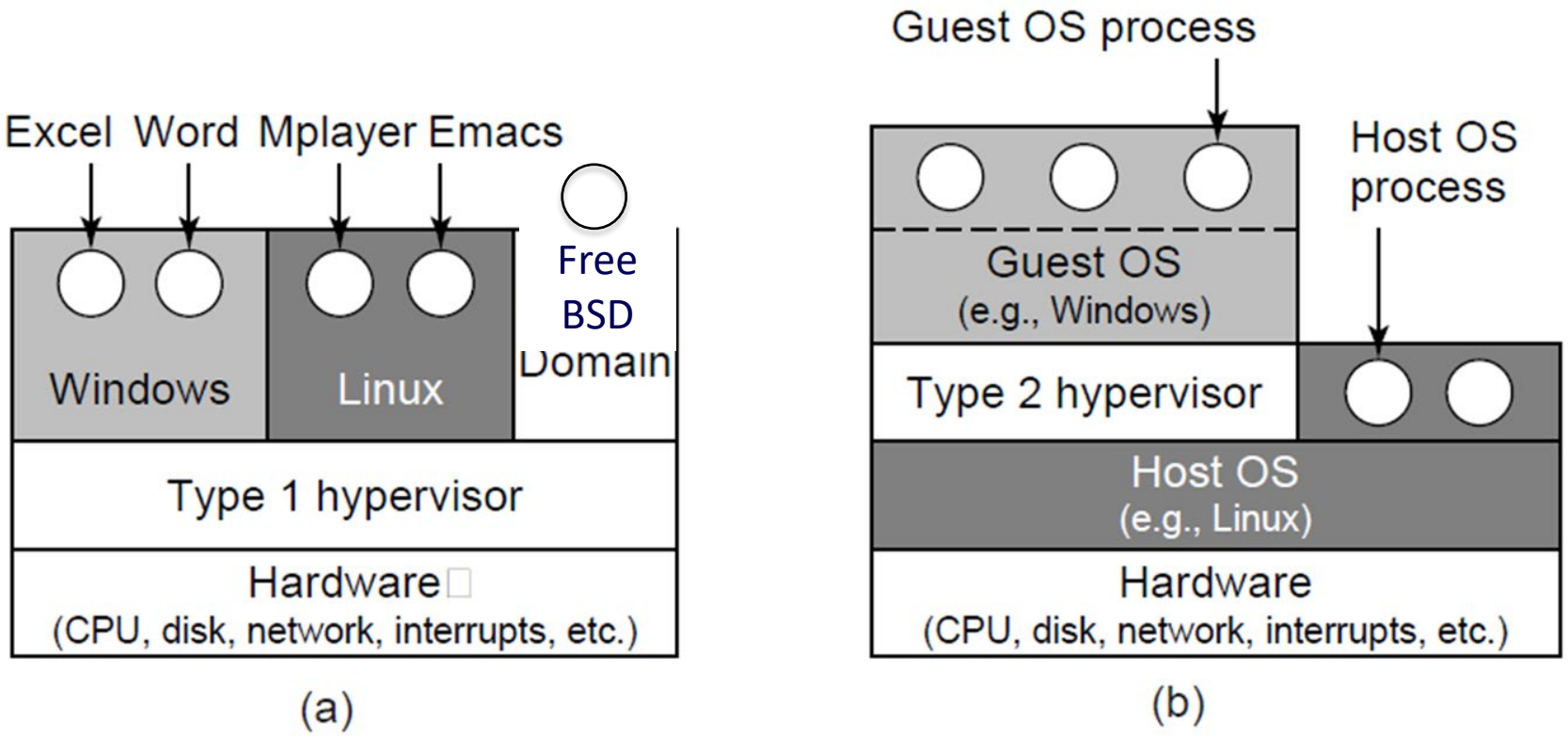
Hypervisors: Do you recall this slide?

Example



Hypervisors

Why do we need these two types?



Location of type 1 and type 2 hypervisors.

Hypervisors: Benefits and Features

- Run multiple, different OSes on a single machine
- Host system protected from VMs, VMs protected from each other
  - i.e., A virus less likely to spread
  - Sharing is provided though via shared file system volume, network communication
- Freeze/suspend running VM
  - Then can move or copy somewhere else and resume
  - Snapshot of a given state, able to restore back to that state
  - Some VMMs allow multiple snapshots per VM
  - Clone by creating copy and running both original and copy

Hypervisors: Benefits and Features

- **Templating** – create an OS + application VM, provide it to customers, use it to create multiple instances of that combination
- **Live migration** – move a running VM from one host to another!
  - No interruption of user access
- All those features taken together -> **cloud computing**
  - Using APIs, programs tell cloud infrastructure (servers, networking, storage) to create new guests, VMs, virtual desktops

Hypervisors: Implementation

- So far, so good. Sounds a simple thing to do.
- Did we mention type 1 hypervisor has something similar to Operating System (OS)?

Hypervisors: Implementation

- Note that OS gives you the access to hardware resources: CPU, memory, I/O.
- Indeed, some type 1 hypervisors look like a microkernel.

- Let us do a quick recap on OSes.

Operating System Basics

- Processes: running programs
- CPU operation mode: user mode vs kernel mode
- OS structures: monolithic system, layered system vs microkernel

Processes

A program runs as a or multiple processes.

What is a process

- A program in execution
- An abstraction of a running program
- Associated with an address space and set of resources
- A container that holds all information needed to run the program

Processes

- Has its own memory space. More specifically:
  - Memory, code, data, stack, open files,
  - PC, registers, stack pointer, etc.

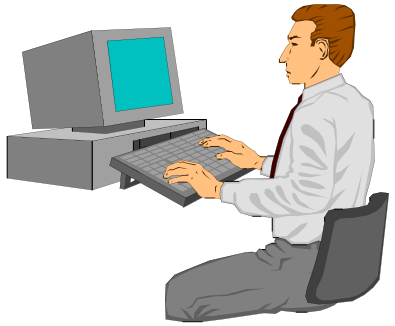
- A program may have more than one process
  - Better utilization of resources
  - Better protection

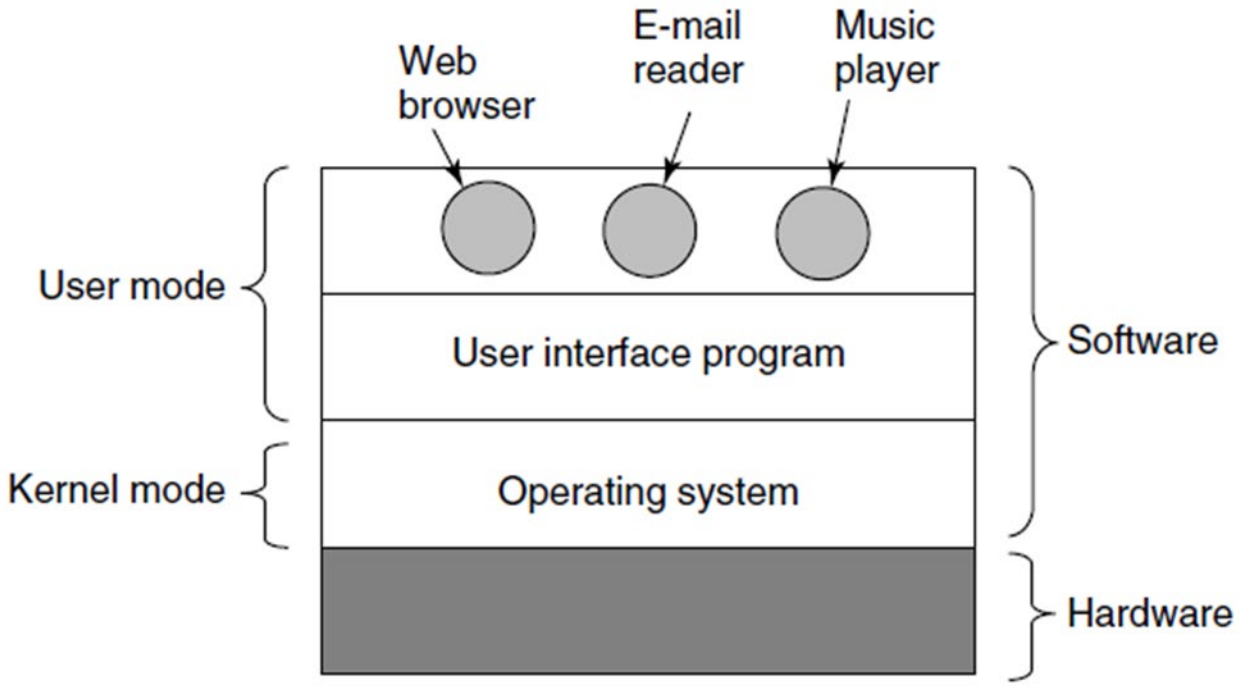
Show me the processes

- Linux: many processes / threads running
  - Try `ps ax'
  - Kernel threads
  - User processes
- Windows: Task Manager

User vs Kernel Mode

Most computers have two modes of operation: user mode and kernel mode.





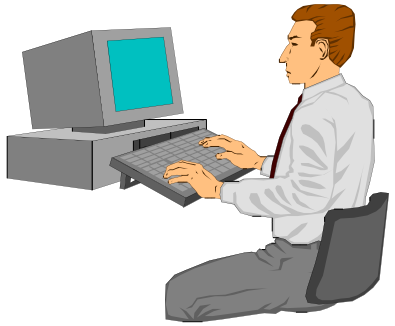
Apps running in user mode. OS running in kernel mode.

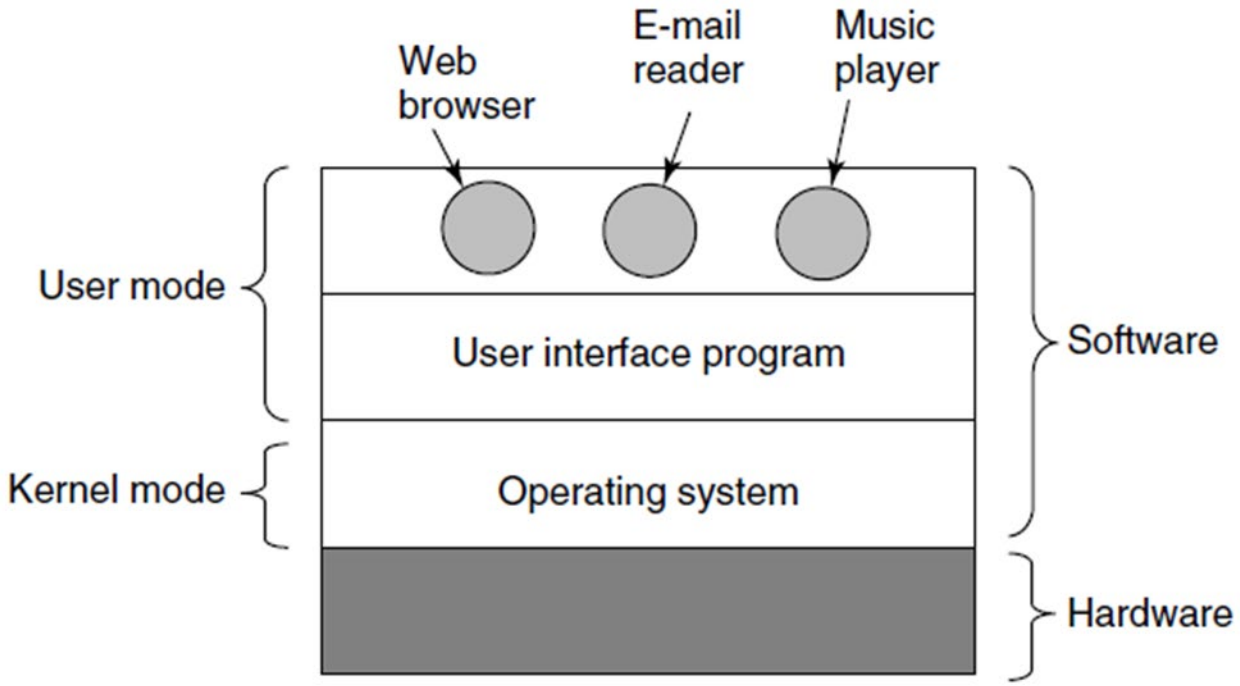
Why do we need these two modes?

- User mode
  - No direct access to hardware
  - No direct access to privileged instructions (I/O, memory management)
  - A crash only affects the involved process
- Kernel mode
  - Full access to hardware
  - Full access to machine instructions.
  - A crash is catastrophic
- Protecting OS and the system from errant users

User vs Kernel Mode: the Switch

Apps (processes) need to make a system call to OS to have access to the hardware, resulting in a mode switch.

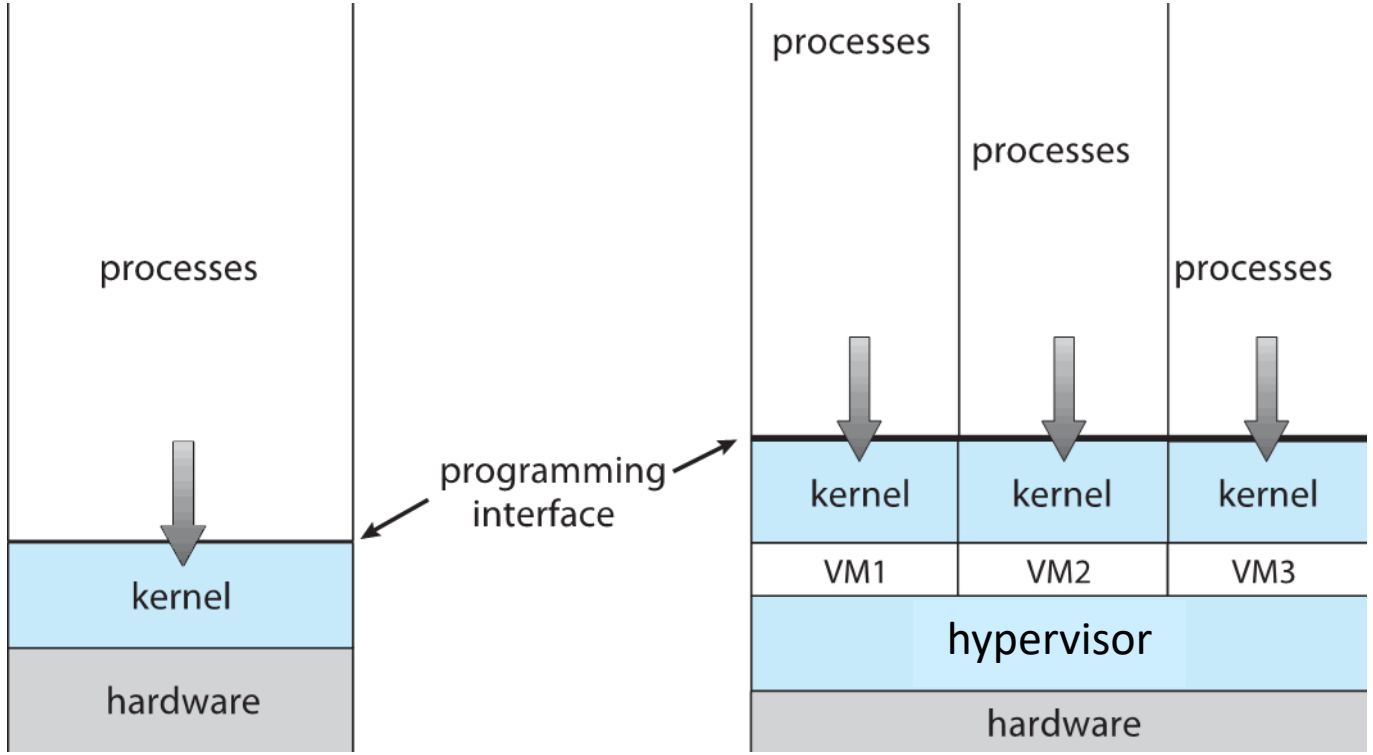




Apps running in user mode. OS running in kernel mode.

User vs Kernel Mode, Hypervisor vs VM

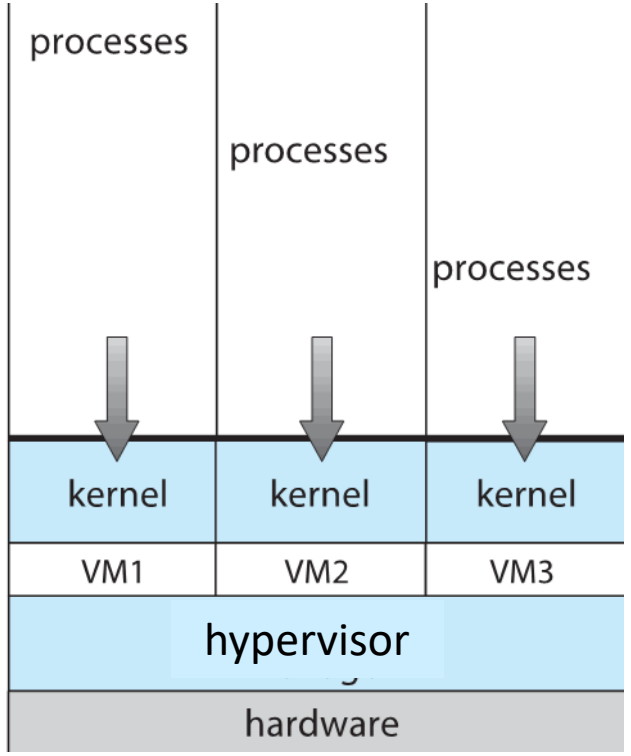
- Hypervisor: kernel mode? Guest VM: user mode?
- How about guest OS?



Non-Virtual Machine vs Virtual Machine

User vs Kernel Mode, Hypervisor vs VM

- Hypervisor: kernel mode
- Guest VM: user mode
  - VM processes: virtual user mode
  - VM OS/kernel: virtual kernel mode
- more on how to implement this later



Virtual Machine

OS structure vs Hypervisor

- Ok, I accept a hypervisor is like an OS
- But different OSes have different structures
  - Monolithic systems
  - Layered systems
  - Microkernel
- Recall the two categories of Type 1 hypervisor

Monolithic systems

- A single kernel containing basically entire functionality of OS
- Written as a collection of procedures, linked together into a single large executable binary program

- Unix/Linux vs KVM

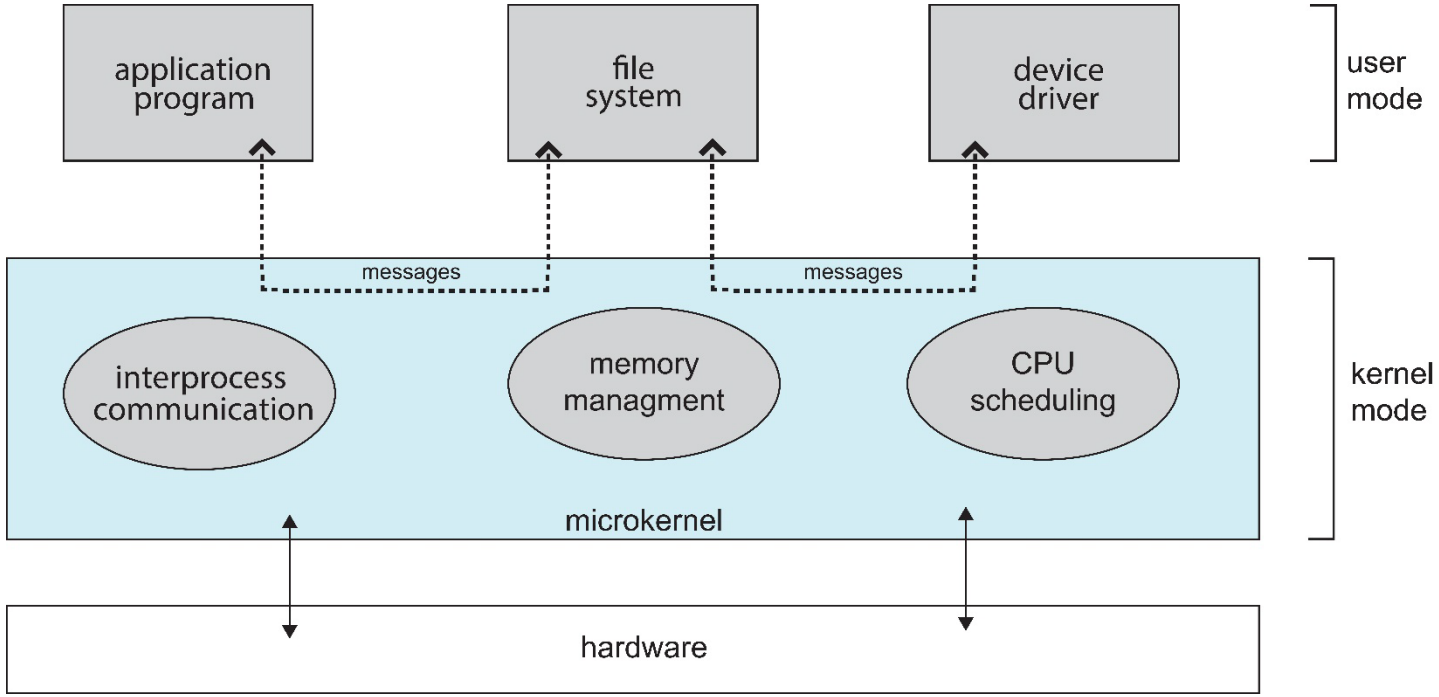
Layered systems

- OS divided into layers, each built on top of lower layers
- The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.

- Windows NT vs Hyper-V

Microkernel

- provides only critical services (e.g. process scheduling and interrupt handling etc.)
  - Moves as much from the kernel into “user” space



- Huawei HarmonyOS, Google Fuchsia vs EXSi

Hypervisor = Microkernel?

- Recall Category 1 of Type 1 hypervisor
- Both are highly reliable and secure, because ….
  - It is small
  - Thousands of lines vs millions of lines of code
- We will talk more about this later ….

Virtualization: what is the problem?

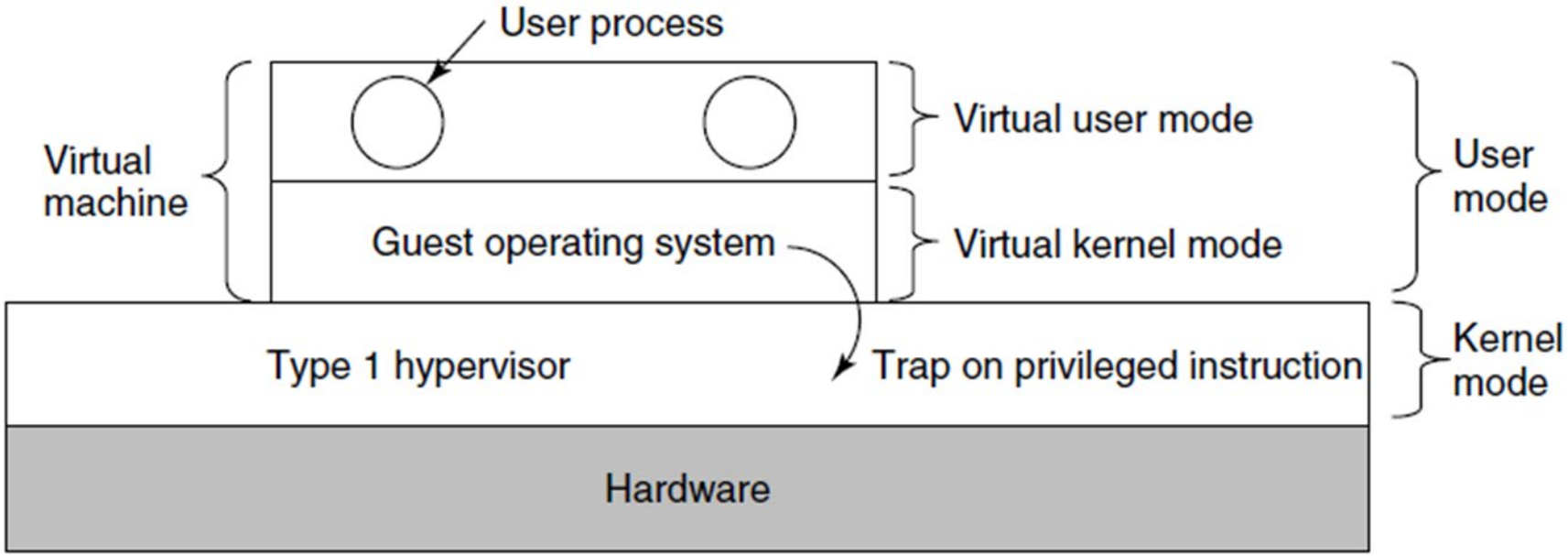
- Guest/VM apps run in user mode as usual
  - It is fine, no problem
- Guest OS runs in kernel mode
  - This is problematic
  - Not safe to let guest kernel run in kernel mode, having direct access to hardware

So,

- VM needs to run in user mode
  - Processes: virtual user mode
  - Kernel: virtual kernel mode
  - Both run in real user mode
- Hypervisor runs in kernel mode

Virtualization: Trap-and-Emulate

How to make it work???



When the operating system in a VM executes a kernel only instruction, it traps to the hypervisor.

Virtualization: Trap-and-Emulate

- VM OS attempting a privileged instruction in user mode causes an error → trap
- Hypervisor gains control, analyses error, executes operation as attempted by VM → emulate
- Returns control to guest in user mode

- Most virtualization products use this technique
  - However, there are performance issues (to be discussed)

Next lecture

- More on virtualization implementation
- Performance issues
- VM live migration and virtual appliances
- OS-level virtualization